

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING-SET-3

Name :Hasmath Nisha M

Register number :192524513

Question 1 — 8-bit Signed Addition&Subtraction (2's Complement)

Problem Statement

A digital computer performs signed integer arithmetic using 8-bit 2's complement representation. Using the numbers -75 and $+38$, perform and analyse both addition ($-75 + 38$) and subtraction ($-75 - 38$), showing the binary conversion, the step-by-step operation, overflow detection, and the final result in both binary and decimal.

Step-by-Step Calculations

Step 1: Convert operands to 8-bit 2's complement

- $+38 \rightarrow 00100110$
- 75 (magnitude) $\rightarrow 01001011 \rightarrow$ invert $\rightarrow 10110100 \rightarrow +1 \rightarrow 10110101 = -75$

Step 2: Addition $(-75) + (+38)$

10110101 (-75)
+ 00100110 ($+38$)

11011011 carry into sign bit = 1, carry out of sign bit = 1

Step 3: Subtraction $(-75) - (+38) = (-75) + (-38)$

- $+38 \rightarrow 00100110 \rightarrow$ invert $\rightarrow 11011001 \rightarrow +1 \rightarrow 11011010 = -38$

10110101 (-75)
+ 11011010 (-38)

10001111 carry into sign bit = 1, carry out of sign bit = 1 (discarded)

Carry-in to MSB (1) = carry-out of MSB (1) $\rightarrow$ no overflow. Result 10001111 has sign bit = 1 (negative); decoding: invert $\rightarrow 01110000$, $+1 \rightarrow 01110001 = 113$, so the value is -113 .

Final Answer

- $-75 + 38 = 11011011_2 = -37$ (decimal), no overflow
- $-75 - 38 = 10001111_2 = -113$ (decimal), no overflow

Analysis / Interpretation

Both results lie well within the representable 8-bit signed range of -128 to $+127$, so no overflow occurs in either case. This is expected: overflow in 2's complement addition can only happen when two numbers of the same sign are added and produce a result of the opposite sign — here, addition mixes a negative and a positive operand, and subtraction produces two same-sign additions (-75 and -38) whose true sum (-113) still fits in the 8-bit range.

Explanation

The processor never truly 'subtracts' — subtraction is converted to addition by negating the second operand (2's complement), which lets the same hardware adder perform both operations. The overflow flag is generated purely by comparing the two carry bits around the sign position, without needing to inspect the operand signs directly, which keeps the arithmetic logic unit (ALU) simple and fast.

Conclusion

Using 8-bit 2's complement arithmetic, $-75 + 38$ evaluates to -37 and $-75 - 38$ evaluates to -113 , and both operations complete without overflow, confirming that 2's complement addition/subtraction with carry comparison at the sign bit is a reliable, hardware-efficient method for signed integer arithmetic.

Question 2 — 8-bit Carry Look-Ahead Adder (CLA)

Problem Statement

A high-performance processor uses an 8-bit Carry Look-Ahead Adder (CLA). For two sample 8-bit operands $A = 01011011$ (91) and $B = 00101110$ (46), analyse how the generate (G) and propagate (P) signals are produced, how all carries are computed simultaneously, and compare the CLA's execution time and hardware complexity against a ripple carry adder (RCA), explaining why CLA is preferred in modern processors.

Step-by-Step Calculations

Step 1: Bit-wise inputs, G_i and P_i ($i = 0$ LSB ... 7 MSB)

i	A _i	B _i	G _i =A _i ·B _i	P _i =A _i ⊕B _i
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	1	1	0
4	1	0	0	1
5	0	1	0	1
6	1	0	0	1
7	0	0	0	0

Step 2: Look-ahead carry equations ($C_0 = 0$)

$C_1 = G_0 + P_0C_0 = 0 + 1 \cdot 0 = 0$
 $C_2 = G_1 + P_1C_1 = 1 + 0 \cdot 0 = 1$
 $C_3 = G_2 + P_2C_2 = 0 + 1 \cdot 1 = 1$
 $C_4 = G_3 + P_3C_3 = 1 + 0 \cdot 1 = 1$
 $C_5 = G_4 + P_4C_4 = 0 + 1 \cdot 1 = 1$
 $C_6 = G_5 + P_5C_5 = 0 + 1 \cdot 1 = 1$
 $C_7 = G_6 + P_6C_6 = 0 + 1 \cdot 1 = 1$
 $C_8 = G_7 + P_7C_7$ (carry-out) $= 0 + 0 \cdot 1 = 0$

Step 3: Sum bits $S_i = P_i \oplus C_i$

i	P _i	C _i	S _i = P _i ⊕C _i
0	1	0	1
1	0	0	0

i	P _i	C _i	S _i = P _i ⊕ C _i
2	1	1	0
3	0	1	1
4	1	1	0
5	1	1	0
6	1	1	0
7	0	1	1

Final Answer

Sum = S₇ S₆ S₅ S₄ S₃ S₂ S₁ S₀ = 10001001₂ = 137, with carry-out C₈ = 0. Check: 91 + 46 = 137. ✓ All eight carries (C₁...C₈) were produced directly and simultaneously from the G/P terms and C₀, not sequentially from stage to stage.

Analysis / Interpretation

Because every C_i depends only on G₀..G_{i-1}, P₀..P_{i-1} and C₀, all carry expressions can be evaluated in parallel by look-ahead logic in roughly 2 gate delays, whereas a ripple carry adder must wait for each carry to physically propagate through all preceding full adders, giving a worst-case delay proportional to the number of bits (n). For 8 bits, RCA delay grows linearly with n while CLA delay stays nearly constant (at the cost of wider, more complex gates for higher bit widths).

Conclusion

The CLA computes all carry signals directly and in parallel using generate/propagate logic, cutting addition time from linear to near-constant with bit-width at the cost of extra gates. This time advantage is why CLA (often organised in hierarchical blocks for wide words) is the standard choice in the ALUs of modern high-performance processors, despite its higher hardware/wiring cost compared with a simple ripple carry adder.

Question 3 — Booth's Algorithm: Multiply +13 × (−6)

Problem Statement

A digital processing unit executes signed multiplication using Booth's algorithm. Multiply +13 by −6, showing the contents of the Accumulator (A), Multiplier (Q), Multiplicand (M), and the Q₋₁ bit after every iteration, and explain how the algorithm reduces the number of add/subtract operations through arithmetic right shifts.

Theory

Booth's algorithm encodes runs of 1s in the multiplier so that a whole string of additions can be replaced by a single subtraction at the start of the run and a single addition at its end, reducing the number of arithmetic operations needed for numbers with long runs of 1s or 0s. Each iteration examines the pair (Q₀, Q₋₁): '10' triggers A = A − M, '01' triggers A = A + M, and '00' or '11' perform no arithmetic. Every iteration ends with an arithmetic right shift (ASR) of the combined (A, Q, Q₋₁) register, and the sign bit is preserved during the shift so that negative partial results remain correctly signed.

Step-by-Step Calculations (5-bit representation, n = 5 iterations)

- Multiplicand M = −6 = 11010, Multiplier Q = +13 = 01101, A = 00000, Q₋₁ = 0

Register Trace

Step	Q0,Q-1	Operation	A (after op)	A,Q,Q-1 (after ASR)
Init	–	–	00000	00000 01101 0
1	1,0	A=A–M	00110	00011 00110 1
2	0,1	A=A+M	11101	11110 10011 0
3	1,0	A=A–M	00100	00010 01001 1
4	1,1	no-op	00001	00001 00100 1
5	0,1	A=A+M	11011	11101 10010 –

Final Answer

Final Product = A,Q (10 bits) = 1110110010₂. Decoding as a signed 2's complement number: invert → 0001001101, +1 → 0001001110 = 78, so the value is –78.

+13 × (–6) = –78 ✓ (matches direct decimal multiplication)

Analysis / Interpretation

The multiplier 01101 (13) contains the bit pattern that Booth's encoding treats as two short runs of 1s, producing one subtraction (step 1, start of a run), one addition (step 2, end of that run), one subtraction (step 3), a no-op (step 4), and a final addition (step 5). Only 3 arithmetic operations were needed across 5 iterations instead of a potential 5 additions/subtractions in an unoptimised shift-add scheme.

Explanation

The arithmetic right shift after every step divides the running partial product by 2 while preserving its sign, correctly repositioning the accumulator relative to the shifting multiplier bits. By replacing consecutive additions for a run of 1s with a single subtraction at the run's start and a single addition at its end, Booth's algorithm reduces the total number of add/subtract cycles, which is especially beneficial for multipliers with long runs of 1s (e.g. numbers close to a power of two).

Question 4 — Restoring vs Non-Restoring Division: 27 ÷ 5

Problem Statement

A binary arithmetic unit supports both restoring and non-restoring division. Divide the binary numbers 27 ÷ 5 using each method, showing the intermediate remainder values, quotient formation, and register contents after every iteration, then compare the two methods on execution steps, computational complexity, and hardware efficiency.

Step-by-Step Calculations

Dividend Q = 27 = 11011, Divisor M = 5 = 00101, A = 00000, n = 5.

Register Trace — Restoring Division

Iter	Shift (A,Q)	A – M	Sign	Action	A after step	Q0 set	Q after step
1	00001,10110	11100 (–)	neg	Restore (A+M)	00001	0	10110
2	00011,01100	11110 (–)	neg	Restore (A+M)	00011	0	01100
3	00110,11000	00001 (+)	pos	Keep	00001	1	11001
4	00011,10010	11110 (–)	neg	Restore (A+M)	00011	0	10010

Iter	Shift (A,Q)	A – M	Sign	Action	A after step	Q0 set	Q after step
5	00111,00100	00010 (+)	pos	Keep	00010	1	00101

Final: Remainder A = 00010 = 2, Quotient Q = 00101 = 5.

Register Trace — Non-Restoring Division

Iter	Shift (A,Q)	Operation	Result A	Sign	Q0 set
1	00001,10110	A – M	11100	neg	0
2	11001,01100	A + M (prev –)	11110	neg	0
3	11100,11000	A + M (prev –)	00001	pos	1
4	00011,10010	A – M (prev +)	11110	neg	0
5	11101,00100	A + M (prev –)	00010	pos	1

Final A = 00010 (positive, so no end correction needed) = Remainder 2; Quotient Q = 00101 = 5.

Final Answer

27 ÷ 5 → Quotient = 5 (00101), Remainder = 2 (00010) — identical result from both methods

Analysis / Interpretation

Both algorithms agree on the result, confirming $27 = 5 \times 5 + 2$. Restoring division performed a corrective addition (restore) in 3 of its 5 iterations (whenever the trial subtraction went negative), i.e. up to 8 arithmetic operations in the worst case (1 subtract + 1 restore per negative step). Non-restoring division performed exactly one addition or subtraction per iteration — only 5 arithmetic operations total, plus at most one final correction — regardless of intermediate signs.

Conclusion

Restoring and non-restoring division both correctly yield $27 \div 5 = 5$ remainder 2, but non-restoring division avoids the extra restore operation on every negative iteration, giving a fixed one-operation-per-cycle structure with lower computational complexity and more predictable timing. This uniformity makes non-restoring division the more suitable choice for high-speed processor design.

Question 5 — IEEE 754 Representation and Floating-Point Addition

Problem Statement

A floating-point processing unit represents real numbers using the IEEE 754 standard. Represent -42.375 in both single-precision (32-bit) and double-precision (64-bit) formats, identifying the sign, exponent, and mantissa fields. Then add -42.375 and $+15.125$ in floating point, explaining exponent alignment, mantissa operation, normalization, rounding, and the effect of precision on the final result.

Theory

IEEE 754 represents a number as $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent}-\text{bias})}$. Single precision uses 1 sign bit, 8 exponent bits (bias 127) and 23 mantissa bits; double precision uses 1 sign bit, 11 exponent bits (bias 1023) and 52 mantissa bits. To add two floating-point numbers, their exponents are first equalised by shifting the mantissa of the smaller-magnitude number right, the (now aligned) mantissas are added or subtracted according to sign, and the result is normalised back to the 1.xxxx form and rounded to fit the available mantissa width.

Step-by-Step Calculations

Step 1: Convert -42.375 to binary and normalise

- $42 = 101010_2$, $0.375 = 0.011_2 \rightarrow 42.375 = 101010.011_2$
- Normalised: 1.01010011×2^5 (sign = 1 for negative)

Step 2: Single precision (32-bit) fields

Sign (1 bit)	Exponent (8 bits) = 5+127=132	Mantissa (23 bits)
1	10000100	01010011000000000000000

Step 3: Double precision (64-bit) fields

[illegible]

Step 4: Convert +15.125 and align exponents for addition

- $15 = 1111_2, 0.125 = 0.001_2 \rightarrow 15.125 = 1111.001_2 = 1.111001 \times 2^3$
- Align to the larger exponent (5): shift mantissa right by 2 places $\rightarrow 0.01111001 \times 2^5$

Step 5: Subtract magnitudes (signs differ: -42.375 is negative, larger magnitude)

$$1.01010011 \times 2^5 \quad (|-42.375|)$$

$$-0.01111001 \times 2^5 \quad (|+15.125|)$$

$$0.11011010 \times 2^5 \rightarrow \text{normalize} \rightarrow 1.101101 \times 2^4$$

Step 6: Normalization&rounding

The subtraction result 0.11011010×2^5 has its leading 1 in the second bit, so the mantissa is shifted left by 1 and the exponent decremented by 1 to restore the 1.xxxx form: 1.101101×2^4 . All bits terminate exactly (0.375 and 0.125 are exact binary fractions), so no rounding error is introduced in this particular case; the sign of the result is negative (from the larger-magnitude operand).

Final Answer

$$-42.375 + 15.125 = -27.25 = -1.101101 \times 2^4$$

In IEEE 754 single precision: sign = 1, exponent = 10000011 ($131 = 4+127$), mantissa = 101101000000000000000000.

Analysis / Interpretation

Because 42.375 and 15.125 are both exactly representable in binary (their fractional parts are sums of negative powers of two), the entire addition — alignment, subtraction, and normalization — is exact, and the IEEE 754 result carries no rounding error. In general, however, aligning the smaller exponent's mantissa by right-shifting can push low-order bits off the end of the finite mantissa field, which is where rounding error would normally be introduced in floating-point addition.

Conclusion

–42.375 is represented exactly in both IEEE 754 single- and double-precision formats, and its floating-point addition with +15.125 — after exponent alignment, mantissa subtraction, and normalization — produces the exact result –27.25, encoded as sign = 1, exponent = 10000011, mantissa =

101101000000000000000000 in single precision, demonstrating the complete IEEE 754 addition pipeline from alignment through normalization.